

Computational-Track

Logical Information Assignment Optimization for Quantum Compilers

Dennis Martin

September 2026

1 Problem Statement

In the computational challenge, we are given a 20-qubit hardware model where each physical qubit has a degree of 1-3, as well as a series of arbitrary operations on one to two qubits in sequence. In the superconducting qubit model, the efficient execution of two-qubit gates imposes the question of how best to assign logical qubits to physical qubit registers, as well as how to optimally use the SWAP gate to move logical states about the hardware graph.

There are essentially three problems which we seek to solve in the computational challenge: 1) what is the best method to position logical qubit states on physical qubits? 2) What is the most expedient method to SWAP qubits such that two distant qubits may execute under a shared gate? 3) What is the optimal scheduling method to execute distinct gates simultaneously, reducing program execution depth and thereby reducing the risk of decoherence?

2 Solution Strategy

In attempting to optimize the solve() function, we looked at a number of approaches (helpfully guided by computational track documentation!) When starting out with the solver design, we observed how "identity" distribution of logical qubits (ie where logical qubits are assigned to physical counterparts bearing the same register) performed poorly, particularly in the number of SWAP gates necessary. A further experiment used multiple random logical assignments and retained the assignment set which resulted in the lowest benchmark score. However, after consulting Li, Ding, and Xie's work "Tackling the Qubit Mapping Problem for NISQ-Era Quantum Devices" which introduced the SABRE assignment and scheduling algorithm, the decision was made to adapt SABRE's methodology to the computational track problem.

The elements of our solver can be described in three broad sections: 1) qubit mapping, 2) composition of the hypothetical set and SABRE heuristics, and 3)

reverse traversal and randomized selection. The method completes sequentially as follows:

1. The distance matrix for each qubit pair is calculated using NetworkX.
2. Based on the gates provided in the program argument, the logical qubit registers which are used are stored as the logical variable.
3. At invocation of the solver function, arbitrary tuning variables (look[*ahead*], weight, iter[*ation*]s, and attempts) are set - these are based on the values used by Li et al.
4. The tries array is populated by 100 initial qubit layouts, starting with the identity layout, and followed by randomized layouts.
5. Each layout is trialed using the attempt() function, and the lowest-scoring layout is passed on.
6. The optimized layout then is improved further using the forward and reverse routing passes as demonstrated in SABRE.
7. The route() method runs the argument program over the optimized layout, emitting single qubit gates and running two qubit gates where seen. This method also runs SWAP gates on separated pairs by listing all potential SWAP candidates which reduce pair distance, running the SABRE heuristic on each candidate, and effecting the cheapest path.
8. After each program execution, the result is scored based on the provided scoring function.
9. Of the 100 starting layouts, the lowest-scoring layout is returned by the solver.

3 Deviations from SABRE

Though the SABRE algorithm is the primary inspiration, and reference of, this solver, there are a number of aspects in which this solver differs from SABRE as shown by Li et al.

1. While SABRE represents all gates ready for execution in a layer, due to the nature of the argument program our solver executes gates sequentially.
2. The heuristic function for SABRE includes a decay term, the purpose of which is to avoid excessive use of SWAP on the same physical qubits. Using a decay function was trialed at values of 0.001 through 0.25, but this had the counterintuitive effect of degrading benchmark performance, resulting in it's omission from our solver.

3. In SABRE, the candidate set includes all qubits which have connections to qubits in the ready gate layer, but this is simplified in our solver to only include edges which would result in distance reduction. This is a concession to ensure program termination in the absence of the decay function.
4. While Li et al use 5 solver attempts with three set traversals, our solver uses 100 attempts and seven traversals, as the higher number of starting layouts improves benchmark performance.

4 Code

```
import random

def solve(program, hardware_graph):

    dist = dict( nx.all_pairs_shortest_path_length(hardware_graph)) #set of distance values

    logical = sorted({q #stores the logical qubits by register
                      for op in program
                      for q in op[1:]})

    #SABRE variables
    #values are taken directly from Li et al.
    look = len(hardware_graph) #Li et al use the size of the graph
    weight = 0.5
    iters = 3
    attempts = 100

    def route(prog, start):
        pos = dict(start) # mapping the initial qubit values to the physical nodes

        occupant = {p: None for p in hardware_graph}
        occupant.update({p: l for l, p in start.items()}) #occupant records which logical q

        partner = {}
        out = []

        for i, op in enumerate(prog):
            if op[0] == "1Q":
                out.append(("1Q", pos[op[1]]))
                continue
```

```

a,b = op[1], op[2]
upcoming = [o[1:] for o in prog[i+ 1:] if o[0] == "2Q"][:look] #taking future q

while not graph.has_edge(pos[a], pos[b]):

    d = dist[pos[a]][pos[b]] #distances between gate qubits
    #candidates are only operations which would reduce distance
    candidates = [(pos[a], n) for n in hardware_graph[pos[a]] if dist[n][pos[b]] < d]

    def cost(swap): #cost function for SABRE derived from Li et al.
        u, v = swap

        #constructing the mapping for candidate SWAP
        moved = {occupant[u]: v, occupant[v]: u}
        where = lambda q: moved.get(q, pos[q])

        future = sum(dist[where(x)][where(y)] for x, y in upcoming) / max(len(upcoming), 1)
        mergeable = partner.get(u) == v and partner.get(v) == u

        return dist[where(a)][where(b)] + weight * future, not mergeable

    #SABRE logic to assign optimal qubits
    u, v = min(candidates, key=cost)
    occupant[u], occupant[v] = occupant[v], occupant[u] #swap function

    for p in (u, v):
        if occupant[p] is not None:
            pos[occupant[p]] = p

    partner.pop(u, None)
    partner.pop(v, None)
    out.append(("SWAP", u, v))

    #gate execution
    p, q = pos[a], pos[b]
    partner[p], partner[q] = q, p
    out.append(("2Q", p, q))

return pos, out

def score(ops):
    layer = {}

```

```

for op in ops:

    if op[0] != "1Q":
        layer[op[1]] = layer[op[2]] = 1 + max(layer.get(op[1], 0), layer.get(op[2], 0))

return sum(op[0] == "SWAP" for op in ops) + 0.5 * max(layer.values(), default=0)

def attempt(physical): #function which simulates reverse circuit traversal

    pos = dict(zip(logical, physical))

    for _ in range( iters ):
        pos = route(program, pos)[0]
        pos = route(program[::-1], pos)[0]
    return pos, route(program, pos)[1]

rng = random.Random(0)
nodes = sorted(graph)
tries = [nodes] + [rng.sample(nodes, len(nodes)) for _ in range(attempts-1)]

return min((attempt(t) for t in tries), key=lambda result: score(result[1]))

```

Gushi Li, Yufei Ding, Yuan Xie, *Tackling the Qubit Mapping Problem for NISQ-Era Quantum Devices*, 2019.